SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – Comparitive Analysis
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Longest Palindromic Subsequence vs LCS		
Student Name:	K HARSHIKA	**Reg. No.:**	192473020
Section / Batch:	CSE-DS	**Date:**	26-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Comparitive Analysis	focuses on applying Dynamic Programming techniques to solve sequence-based optimization problems by finding the Longest Palindromic Subsequence and comparing it with the LCS approach.	Q1

SECTION A – Comparitive Analysis

Code of Conduct

I K HARSHIKA(192473020)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARSHIKAREDDY_____

RUBRICS FOR CALCULATION

Comparitive Analysis

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Longest Palindromic Subsequence vs LCS

Introduction

Dynamic Programming (DP) is commonly used to solve sequence-based problems where a larger problem can be divided into smaller overlapping subproblems. The Longest Palindromic Subsequence (LPS) problem is one such application. A palindromic subsequence is a sequence of characters that reads the same from both directions, while the characters do not necessarily have to be consecutive.

For the given string:

S = "BBABCBCAB"

the objective is to find the longest palindromic subsequence and compare the direct LPS Dynamic Programming approach with the Longest Common Subsequence (LCS) approach.

Longest Palindromic Subsequence

A palindrome reads the same from left to right and from right to left. By selecting appropriate characters from the given string, one possible longest palindromic subsequence is:

BABCBAB

The sequence remains the same when reversed:

BABCBAB

Therefore, the length of the Longest Palindromic Subsequence is:

LPS Length = 7

Another valid subsequence of length 7 is **BACBCAB**. Hence, the given string has a longest palindromic subsequence of length 7.

Dynamic Programming Approach for LPS

In the direct LPS approach, a two-dimensional DP table is used. Let `dp[i][j]` represent the length of the longest palindromic subsequence between index `i` and index `j`.

The base condition is that every single character is a palindrome of length 1. Therefore:

```
dp[i][i] = 1
```

If the characters at positions `i` and `j` are equal, both characters can be included in the palindrome. Hence:

```
dp[i][j] = 2 + dp[i+1][j-1]
```

If the characters are different, we cannot include both of them as the boundary characters of the same palindrome. Therefore, we take the maximum value obtained by excluding either one:

```
dp[i][j] = max(dp[i+1][j], dp[i][j-1])
```

By filling the DP table from smaller substrings to larger substrings, the final value `dp[0][n-1]` gives the answer. For the string **BBABCBCAB**, the result obtained is **7**.

The algorithm works because every palindrome is constructed from smaller palindromic subsequences. If the two boundary characters are the same, they extend the smaller palindrome inside them. If they are different, the optimal solution must exist after removing one of the boundary characters. Thus, the recurrence considers all necessary possibilities and guarantees an optimal solution.

LCS Approach for Finding LPS

The Longest Palindromic Subsequence problem can also be related to the Longest Common Subsequence problem. The basic idea is to compare the original string with its reverse.

The original string is:

S = BBABCBCAB

Its reverse is:

R = BACBCBABB

Now, the Longest Common Subsequence between these two strings is computed using Dynamic Programming.

The LCS recurrence is:

If the characters match:

```
dp[i][j] = 1 + dp[i-1][j-1]
```

If the characters do not match:

```
dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

The LCS length obtained for the given string and its reverse is **7**, which corresponds to the length of the Longest Palindromic Subsequence.

Thus:

LPS(BBABCBCAB) = 7

The LCS-based approach provides an alternative way to solve the problem by transforming a single-sequence palindrome problem into a two-sequence comparison problem.

Comparison Between LPS and LCS Approaches

Both approaches use Dynamic Programming because both problems have overlapping subproblems and optimal substructure. In both cases, a two-dimensional table is created and the solution to larger problems is obtained using the results of smaller problems.

However, the main difference lies in the meaning of the DP state and the recurrence relation. In the direct LPS approach, the algorithm works on two positions within the same string. The recurrence compares the first and last characters of a substring. When they are equal, two characters are added to the palindrome.

In the LCS approach, two different sequences are compared: the original string and its reverse. When characters match, only one character is added to the common subsequence.

Therefore, the main difference can be summarized as follows:

Aspect	Direct LPS	LCS Approach
Input	One string	Original string and reverse
DP State	Substring from i to j	Prefixes of two strings
Match Case	2 + inner palindrome	1 + diagonal value
Purpose	Find palindrome directly	Find common subsequence
Time Complexity	$O(n^2)$	$O(n^2)$
Space Complexity	$O(n^2)$	$O(n^2)$

Time and Space Complexity

For a string of length n, the LPS DP algorithm creates an $n \times n$ table. Each entry is calculated only once and requires constant time.

Therefore:

Time Complexity = $O(n^2)$

Space Complexity = $O(n^2)$

Similarly, the LCS approach compares two strings of length n and creates an $n \times n$ DP table. Hence:

Time Complexity = $O(n^2)$

Space Complexity = $O(n^2)$

Thus, both approaches have similar theoretical performance.

For long strings, the major challenge is memory usage because the DP table grows quadratically with the length of the input. For very large strings, storing the complete table may become impractical. If only the length of the solution is required, space optimization techniques can reduce the memory requirement from $O(n^2)$ to $O(n)$. However, the time complexity generally remains $O(n^2)$.

Insights on Sequence-Based Dynamic Programming

This problem demonstrates several important concepts of Dynamic Programming. First, the correct selection of a DP state is essential. The direct LPS approach uses intervals within a single string, whereas the LCS approach uses prefixes of two sequences. Second, the problem shows that one problem can sometimes be transformed into another well-known problem. By reversing the original string and applying the LCS concept, the LPS problem can be analyzed from a different perspective.

Sequence-based DP problems generally involve identifying smaller subproblems, defining suitable recurrence relations, and storing intermediate results to avoid repeated computation. Similar ideas are used in problems such as Longest Common Subsequence, Edit Distance, Sequence Alignment, and Longest Increasing Subsequence.

CODE

```
# Longest Palindromic Subsequence and LCS Comparison


def longest_palindromic_subsequence(s):
    n = len(s)


    # Create DP table
    dp = [[0] * n for _ in range(n)]


    # Every single character is a palindrome of length 1
    for i in range(n):
        dp[i][i] = 1


    # Fill the table
    for length in range(2, n + 1):
```

```
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                if length == 2:
                    dp[i][j] = 2
                else:
                    dp[i][j] = 2 + dp[i + 1][j - 1]
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

    return dp[0][n - 1], dp



def longest_common_subsequence(s1, s2):
    m = len(s1)
    n = len(s2)

    # Create DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):

            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
```

```
            else:
                dp[i][j] = max(dp[i - 1][j],
                               dp[i][j - 1])

    return dp[m][n]



# Given string
s = "BBABCBCAB"

# Direct LPS approach
lps_length, lps_table = longest_palindromic_subsequence(s)

# LCS approach
reverse_s = s[::-1]
lcs_length = longest_common_subsequence(s, reverse_s)



# Display results
print("Original String:", s)
print("Reversed String:", reverse_s)

print("\n--- Direct LPS Approach ---")
print("Longest Palindromic Subsequence Length:", lps_length)

print("\n--- LCS Approach ---")
print("LCS Length between String and Reverse:", lcs_length)
```

```
print("\n--- Comparison ---")

if lps_length == lcs_length:

    print("Both approaches give the same result:", lps_length)

else:

    print("Results are different")
```

OUTPUT

```
Original String: BBABCBCAB
Reversed String: BACBCBABB

--- Direct LPS Approach ---
Longest Palindromic Subsequence Length: 7

--- LCS Approach ---
LCS Length between String and Reverse: 7

--- Comparison ---
Both approaches give the same result: 7
```

Conclusion

For the string BBABCBCAB, the Longest Palindromic Subsequence has a maximum length of 7. One possible LPS is BABCBAB. The problem can be solved directly using an interval-based Dynamic Programming approach or analyzed using the Longest Common Subsequence approach with the reversed string. Both approaches have a time complexity of $O(n^2)$ and a space complexity of $O(n^2)$.

The comparison shows that although LPS and LCS use similar Dynamic Programming principles, their state definitions and recurrence relations are different. The problem also highlights the importance of problem transformation, optimal substructure, and overlapping subproblems in designing efficient algorithms for sequence-based problems.